Architecture
Project 3: WordlePlus System Development — EECS 581: Software Engineering II
Fall 2026 · Professor Hossein Saiedian

Team Number: Group 9

Team Members:
- Alexander Wichern
- Dhruvi Patel
- Joseph Ubanda Ruiz
- Niles Chen
- Rahul Nesan Jawahar Nesan
- Tram Dinh
- Umair Chaudhry

Project Name: WordlePlus

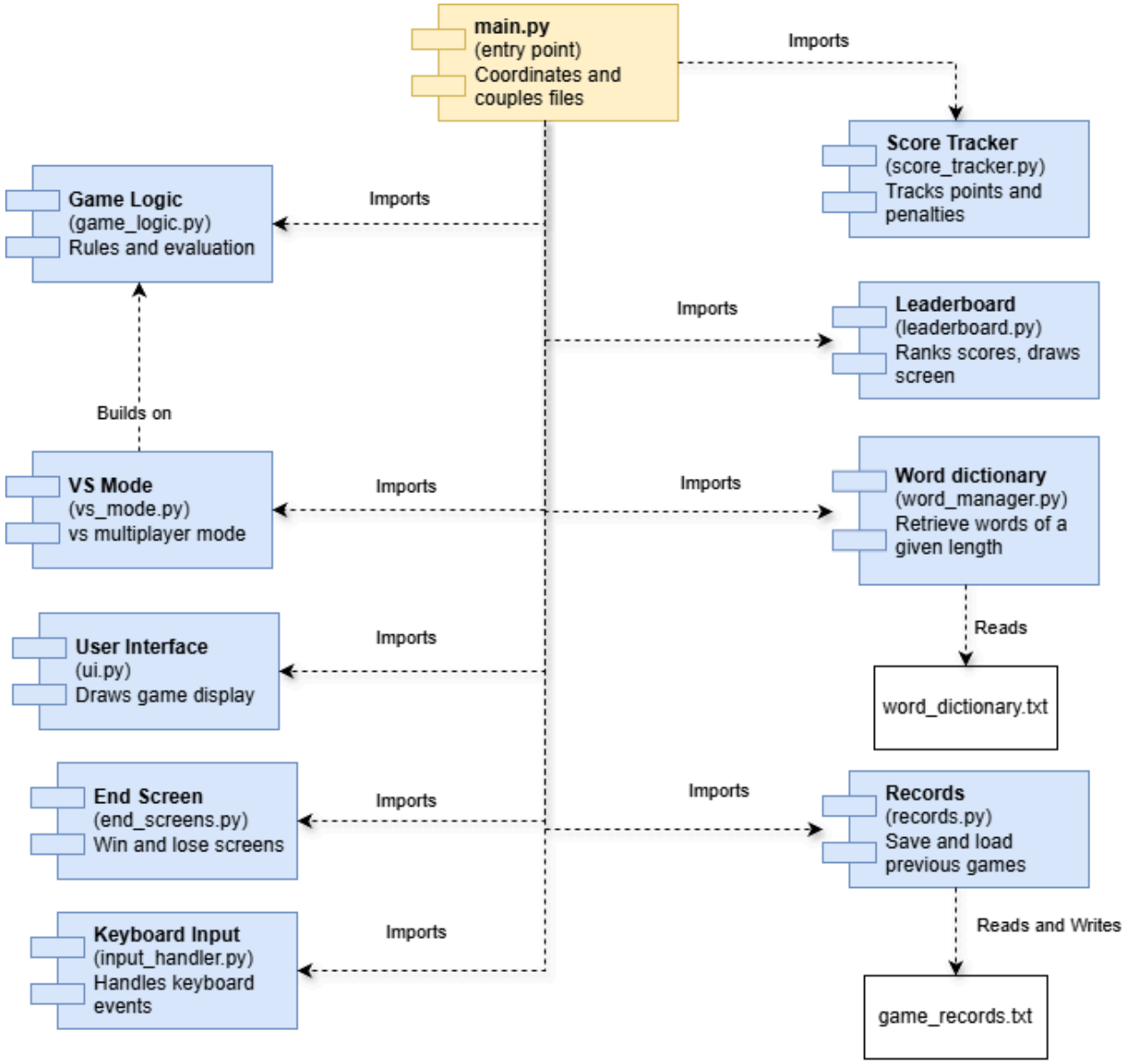
1. Project Synopsis (1–25 words)

WordlePlus expands the classic word-guessing game with selectable difficulty, hints, score tracking, a leaderboard, and vs multiplayer mode.

2. Architecture Description (500–1000 words, 3 distinct UML diagrams).

https://drive.google.com/drive/folders/1QiukLgqpdGL2NheNzzkdCRast9dNpt8g?usp=sharing

WordlePlus is a word-guessing game built in Python. Its components are separated into their own Python files and coupled together in `main.py`. The primary components are the user interface, game logic, keyboard input, score tracker, leaderboard, win and lose screens, and VS mode. This separation lets team members develop parts in parallel and lets later features be added without rewriting the core game.The system is described through three views: a component diagram of the file structure, a class diagram of the main game objects, and a sequence diagram of what happens when a player submits a guess.



The first UML diagram shows the program's files and how they connect. `main.py` is the entry point.

- `main.py` creates the window, runs the main game loop, tracks which screen is active (start, difficulty, playing, win, lose, statistics, leaderboard, or VS), and imports every other file, passing data between them so that most files do not depend on each other directly. The `vs_mode.py` is an exception as it builds on the default game logic in the `game_logic.py`.

- `ui.py` draws the start screen, difficulty screen, game board, tiles, current score, and statistics screen.

- `input_handler.py` captures keyboard events such as typed letters, backspace, and Enter, and reports them to main.py as simple actions. For example hitting the ENTER key will make the program check the entered word from the user.

- `game_logic.py` holds the rules: it stores the board and target word, evaluates guesses, detects wins and losses, resets the game for a retry, and handles the vowel hint.

- `word_manager.py` reads word_dictionary.txt, selects a random target word of the required length, and checks whether a guess is a valid dictionary word.

- `score_tracker.py` calculates points for correctly placed letters and fast wins and applies the hint penalty.

- `records.py` reads and writes game_records.txt, saving each completed game and loading previous games, so statistics persist between sessions.

- `leaderboard.py` ranks those records by score, breaking ties with the fewest guesses, and draws the leaderboard screen.

- `end_screens.py` draws the win and lose screens and their retry buttons.

- `vs_mode.py` manages two players and alternating turns.



The second UML diagram is a use case diagram showing what a player can do in WordlePlus.

- "Start Game" begins a new game. It includes "Select difficulty," where the player chooses the word length and board size, and "Enter Player Name," where the player enters the name used for statistics and the leaderboard.

- "Submit guess" is how the player enters a word and submits it with the Enter key. It includes "Validate guess," which checks that the word is complete and in the dictionary, and "Update color tiles for each letter," which marks each letter green, yellow, or grey.

- "Use hint" is an optional extension of "Submit guess" that reveals which of the vowels A, E, I, O, and U are in the word. It can be used once per game and applies a score penalty.

- “Win game” and “Lose game” are extensions of “Submit guess” that happen only under a condition. A win happens when the guess matches the target word, and a loss happens when all guesses are used. Each ends the game and shows the matching screen.

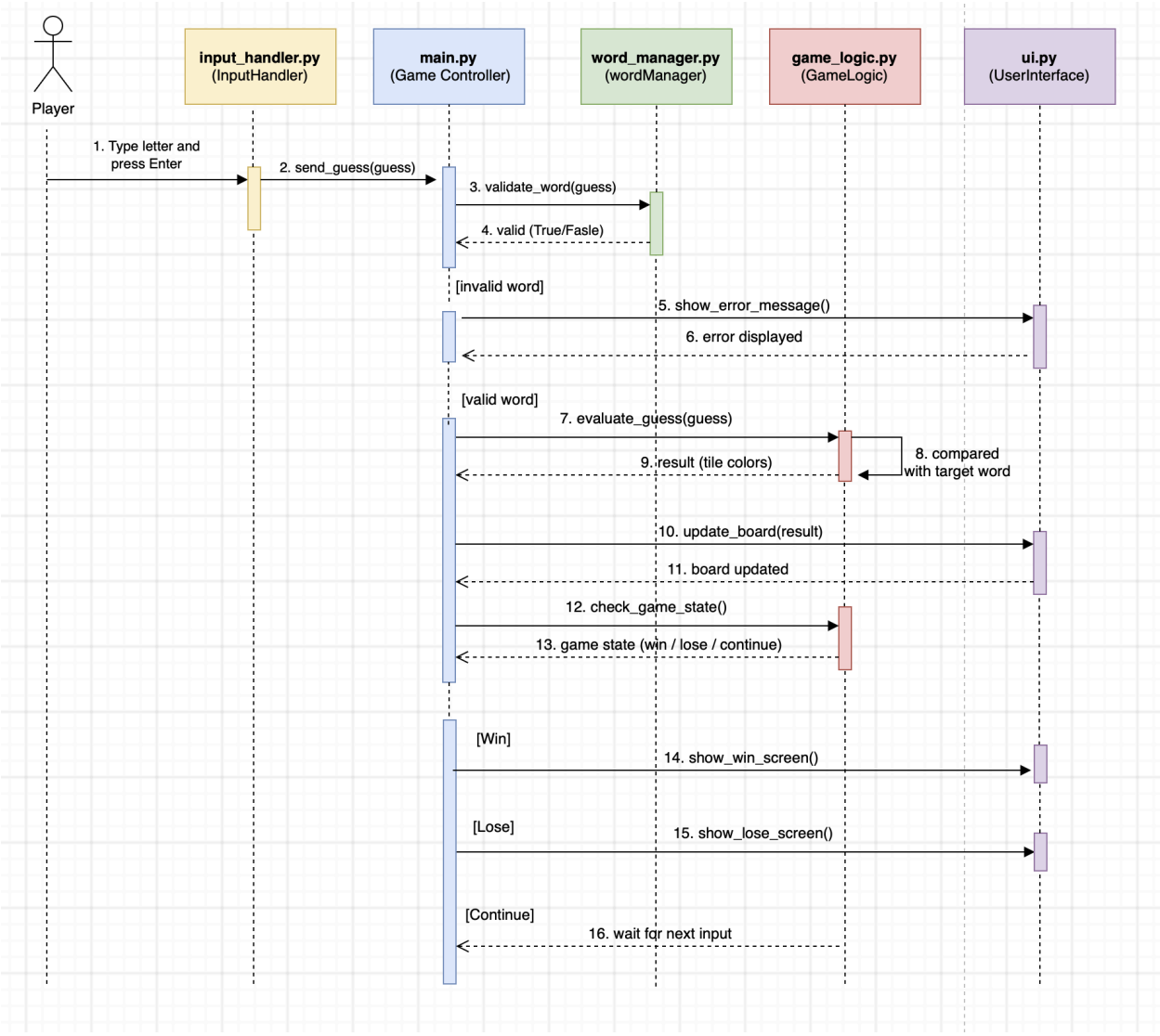
- “Retry game” lets the player start another game after a win or a loss, which resets the board and chooses a new word.

- “View Leaderboard” shows saved games ranked by score, with ties broken by the fewest guesses.

- “View statistics” shows the player’s personal totals, such as games played, win rate, and average guesses.

- “Play VS mode” lets two players take turns guessing, and the first player to guess the word wins.

The third UML diagram



The third UML diagram is a sequence diagram showing how the program's components interact when a player submits a guess.

- Player and input_handler.py: The player types a word and presses Enter. input_handler.py captures the keyboard input and sends the guess to main.py.
- main.py and word_manager.py: main.py sends the guess to word_manager.py to check whether it is a valid word. If the word is invalid, the program shows an error message and waits for another guess.
- game_logic.py: If the word is valid, main.py sends the guess to game_logic.py, which compares it with the target word and determines the correct color for each letter.

- ui.py: After the guess is evaluated, main.py sends the results to ui.py to update the game board. Green means the letter is in the correct position, yellow means it is in the wrong position, and grey means the letter is not in the word.

- Game Status: main.py checks the game status through game_logic.py. If the player guesses the correct word, the game displays the win screen. If the player uses all five guesses, the game displays the lose screen. Otherwise, the player can continue guessing.